

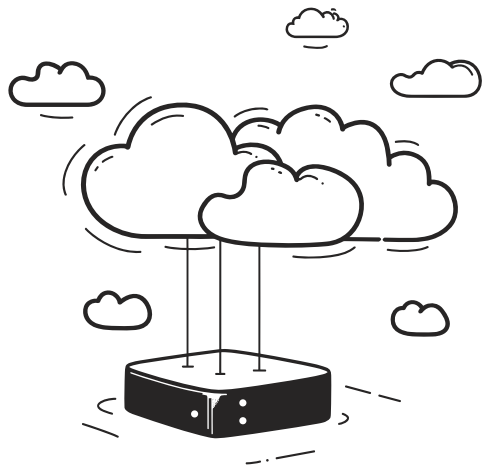
Teoria R4 · Contracte d'API, HTTP i respostes

De l'aplicació interna de R3 a un servei reusable i consumible per qualsevol client.

DAW · R4



De R3 a R4: el canvi de paradigma



Fins ara, a R3...

L'aplicació generava HTML al servidor i el navegador el mostrava directament. El servidor **sabia** qui consumia la resposta.

A partir de R4...

El servidor exposa **dades estructurades** (JSON). Qualsevol client —navegador, app mòbil, altre servei— pot consumir-les sense conèixer el codi.



Què és el contracte d'API?

Definició

Acord formal entre el servei i el consumidor: quina URL cridar, com enviar les dades i quina resposta esperar.

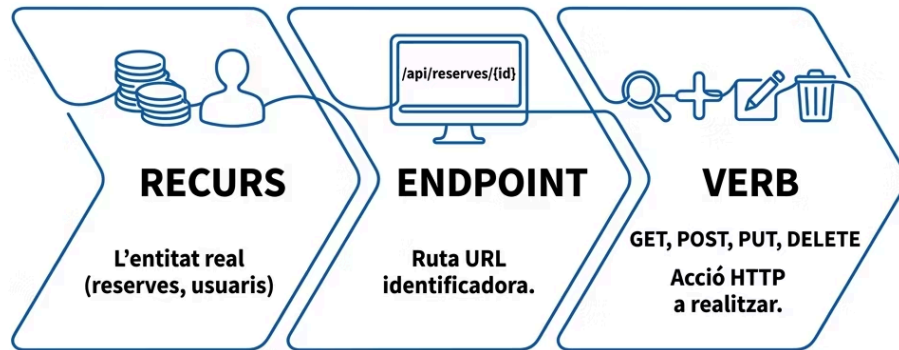
Regla d'or

El consumidor **no ha de llegir el codi** per saber com usar l'API. El contracte ho explica tot.

Consistència

La resposta real sempre ha de coincidir amb la documentació del contracte. Si canvia, el contracte es trenca.

Recurs, Endpoint i Verb



Un exemple concret

GET /api/reserves/{id}

- **Recurs:** una reserva del projecte
- **Endpoint:** la ruta que la identifica
- **Verb:** GET → volem llegir, no modificar

i Cada endpoint ha de representar una **operació o recurs real** del projecte, no una funció interna del codi.

La Request: tot el que envia el client

1

Ruta i paràmetres

`/api/reserves/{id}` — el recurs concret. Query params: `?pagina=2`

2

Capçaleres

Metadades de la petició: `Content-Type: application/json,`
`Authorization: Bearer ...`

3

Cos (body)

En POST/PUT: les dades en JSON.
En GET/DELETE: normalment buit.

La Response: el que retorna el servidor

Resposta 200 OK

```
{  
  "id": 42,  
  "usuari": "maria@daw.cat",  
  "data": "2025-06-10",  
  "estat": "confirmada"  
}
```

Codi d'estat + cos JSON + capçaleres de resposta.

Comparació HTML vs JSON

R3 (HTML)	R4 (JSON)
Renderitza directament	Dades estructurades
Només navegador	Qualsevol client
Difícil de reutilitzar	Fàcil d'integrar
CSS inclòs	Sense presentació

Codis HTTP més habituals

2xx · Èxit

- **200** OK
- **201** Created
- **204** No Content

4xx · Error del client

- **400** Bad Request
- **401** Unauthorized
- **403** Forbidden
- **404** Not Found

5xx · Error del servidor

- **500** Internal Server Error
- **503** Service Unavailable



El codi d'estat **no és opcional**: forma part de la resposta funcional i el client el llegeix abans del cos.



Errors: dissenyats, no improvisats

Un bon error evita que el client haja d'*endevinar* què ha passat. Ha d'incloure codi, missatge i, si cal, detall.

400 — Dades invàlides

```
{ "error": "El camp 'data' és obligatori" }
```

401 — No autenticat

```
{ "error": "Token no proporcionat o invàlid" }
```

404 — Recurs no trobat

```
{ "error": "Reserva 42 no existeix" }
```


Exemple de contracte: GET /api/reserves/{id}

Camp	Valor
Mètode	GET
URL	/api/reserves/{id}
Capçalera	Authorization: Bearer {token}
Body	Cap
Resposta OK	200 + JSON amb les dades de la reserva
Errors previstos	401 (no autenticat), 403 (sense permís), 404 (no trobada)



Definir el contracte **abans de programar** evita sorpreses i facilita el treball en equip.

Diagrama del flux complet



El contracte actua com a garantia entre les dues bandes: el que el servidor promet i el que el client espera han de ser sempre el mateix.

Check final abans d'implementar

1 Recurs real

Quin recurs real del projecte exposa aquest endpoint?

2 Contracte complet

Tens definits el mètode, la URL, el request i la resposta esperada?

3 Codis d'estat

Quin codi HTTP correspon al cas correcte i quins als errors?

4 Errors previstos

Has pensat en tots els casos que poden fallar *abans* de programar?

5 Llegibilitat

Una altra persona podria consumir l'endpoint sense llegir el codi?